

CISP 400

Object Oriented Programming in C++

Dr. Fowler



F O L S O M L A K E C O L L E G E
EL DORADO CENTER | RANCHO CORDOVA CENTER

**WHEN YOUR CODE IS A COMPLETE
MESS BUT STILL GETS THE JOB DONE.**



Last Time

Agenda

Ch 7 - Object Oriented Programming

7.1 Abstract Data Types

7.2 Object-Oriented
Programming

7.3 Introduction to Classes

7.4 Creating and using Objects

7.5 Defining Member Functions

7.6 Constructors

7.7 Destructors

7.8 Private member Functions

7.9 Passing Objects to Functions

7.10 Object Composition

7.11 SKIP

7.12 Structures

7.13 More on Enumerated Data
Types

8.13 Arrays of Objects

Theme for Today

“Character Arrays”

Chapter 12

Strings

Agenda

Ch 12 - Strings

12.1 C-Strings

12.2 Library Functions for
Working with C-Strings

12.3 Conversions
between Numbers and
Strings

12.4 Writing Your Own C-
String Handling Functions

12.5 More about the C++
String Class

Agenda

Ch 12 - Strings

12.1 C-Strings

12.2 Library Functions for Working with C- Strings

12.3 Conversions between Numbers and Strings

12.4 Writing Your Own C-String Handling Functions

12.5 More about the C++ String Class

Big Idea

- In C++, you can't get away from C-Style strings.
- But rather than fight it, master it and use their strengths to hone your coding ninja skills.

Related to Previous Chapters ...

- We needed to know pointers to discuss this.
- Skills you learn here will help you understand upcoming chapters.

12.1 C-Strings

C++ Strings

- `#include <iostream>`
- Using them all along
- Very useful - objects

C strings

- very different from C++ strings
 - a collection of characters in memory
 - terminated with a `\0`

Index	0	1	2	3	4	5
Variable	H	e	l	l	o	\0
Address	0x23451	0x23452	0x23453	0x23454	0x23455	0x23456

Variable type of a C-String is a pointer to char.


```
1  //3-24.cpp -- mix C & C++ strings
2  // Buffer overflow no problem.
3  #include <iostream>
4  using namespace std;
5
6  int main() {
7      const int SIZE = 12;
8      char name[SIZE];
9
10     cout << "Enter your first name: ";
11     cin >> name;
12     cout << "Hello, " << name << endl;
13
14     return 0;
15 }
```



We need to be mindful of buffer overflow/overrun with C strings.

Not an issue with C++ Strings.

Stop buffer overrun

```
1  //3-26.cpp -- limit data entry size
2  // p.127
3  #include <iostream>
4  #include <iomanip>    // setw method only
5  using namespace std;
6
7  int main() {
8      const int SIZE = 5; // note what happens to \n
9      char word[SIZE];
10
11      cout << "Enter a word: ";
12      cin >> setw(SIZE) >> word;
13      cout << "You entered, " << word << endl;
14
15      cout << "Enter another word: ";
16      cin.width(SIZE);
17      cin >> word;
18      cout << "You entered, " << word << endl;
19
20      return 0;
21      // remember, cin doesn't do whitespace
22  }
```

We also can read in an entire line of text to the C string ...

```
1 //3-28.cpp -- getline for c string
2 // p.129, can handle whitespace
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     const int SIZE = 5;
8     char input[SIZE];
9
10    cout << "Enter input: ";
11    cin.getline(input, SIZE);
12    cout << "You entered: " << input << endl;
13
14    return 0;
15 }
```

This is NOT the same as `getline()`;



Ways to access C-Strings

1. Hard Code string literals
2. Programmer defined char arrays
 - for all I/O operations.
3. Pointers to char
 - Allow us to change the string, but use the same variable.



```
1  // 12-1.cpp -- String literals for C strings
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      const char* p = nullptr;
8      const char* q = nullptr;
9      p = "Hello ";
10     q = "Bailey";
11
12     cout << p << q << endl;
13     cout << p << " is stored at " << long(p) << endl;
14     cout << q << " is stored at " << long(q) << endl;
15
16     // Treat a string literal like a pointer
17     cout << "String literal stored at " << long("here");
18
19     return 0;
20 }
```

```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      const char* p = nullptr;
7      const char* q = nullptr;
8      const char* t = nullptr;
9      p = "Hello ";
10     q = "Bailey";
11
12     cout << p << q << endl;
13     cout << p << " is stored at " << long(p) << endl;
14     cout << q << " is stored at " << long(q) << endl;
15
16     // Treat a string literal like a pointer
17     const char* ch = "String literal stored here ";
18     cout << ch << &ch << endl;
19
20     // mixing ptrs
21     t = p;
22     cout << t << endl;
23     t = q;
24     cout << t << endl;
25
26     return 0;
27 }

```


C-String General Rules

No bounds checking!

Relational operators (`<=`, `<`, `>`, `>=`, `!=`, `==`) do not work!

C-string and C++ string Comparison

Feature	C-String	C++ String (<code>std::string</code>)
Memory Allocation	Manual (static, stack, or dynamic)	Automatic, dynamic management
Bounds Checking	None (prone to buffer overflows)	Automatic (prevents overflow)
Resizing	Manual (allocate, copy, free memory)	Automatic resizing
Null Termination	Required (<code>\0</code> indicates end)	Not needed (managed internally)
Efficiency	Fast but risky (no safety features)	Slight overhead but safer
Convenience	Manual operations (copy, concat)	Operators and functions for ease

12.2 Library Functions for Working with C-Strings

#include <cstring>

Ton of functions for dealing with C-Strings.

<http://www.cplusplus.com/reference/cstring/>

Functions

Copying:

memcpy	Copy block of memory (function)
memmove	Move block of memory (function)
strcpy	Copy string (function)
strncpy	Copy characters from string (function)

Concatenation:

strcat	Concatenate strings (function)
strncat	Append characters from string (function)

Comparison:

memcmp	Compare two blocks of memory (function)
strcmp	Compare two strings (function)
strcoll	Compare two strings using locale (function)
strncmp	Compare characters of two strings (function)
strxfrm	Transform string using locale (function)

Searching:

memchr	Locate character in block of memory (function)
strchr	Locate first occurrence of character in string (function)
strcspn	Get span until character in string (function)
strpbrk	Locate characters in string (function)
strrchr	Locate last occurrence of character in string (function)
strspn	Get span of character set in string (function)
strstr	Locate substring (function)
strtok	Split string into tokens (function)

Other:

memset	Fill block of memory (function)
strerror	Get pointer to error message string (function)
strlen	Get string length (function)

Macros

NULL	Null pointer (macro)
-------------	-----------------------

Types

size_t	Unsigned integral type (type)
---------------	--------------------------------

What's My Output?

Output12-20x.cpp

Exercise Goals

- Review of code syntax
- Check understanding of syntax

Desk Check Exercise

- Pair up
- Manually determine the output of the following code?
DO NOT TYPE THIS IN AND EXECUTE IT.
- We'll spend about 10 minutes on this, then go over it.


```
1 // Output12-20x.cpp -- what's the output?
2 // Gladdis p.846 # 20, 22, 23, 24
3 #include <iostream>
4 #include <string>
5 #include <cstring>
6 #include <iostream>
7 using namespace std;
8
9 int main() {
10     cout << ("hello")[1] << endl;
11     cout << *("C++ is fun" + 5) << endl;
12     cout << string("fantastic").size() << endl;
13     cout << std::boolalpha
14         << static_cast<bool>(strcmp("a", "b")) << endl;
15
16     return 0;
17 }
```



```

1 // Output12-20x.cpp -- what's the output?
2 // Gladdis p.846 # 20, 22, 23, 24
3 #include <iostream>
4 #include <string>
5 #include <cstring>
6 #include <iostream>
7 using namespace std;
8
9 int main() {
10     cout << ("hello")[1] << endl;
11     cout << *("C++ is fun" + 5) << endl;
12     cout << string("fantastic").size() << endl;
13     cout << std::boolalpha
14         << static_cast<bool>(strcmp("a", "b")) << endl;
15
16     return 0;
17 }

```

```

e
s
9
true

```

12.3 Conversions between Numbers and Strings

“The C++ libraries provide functions and classes that can be used to convert a string representation of a number to a numeric form and vice versa.”

–p.829

Can't process strings like other numeric types.

We need to convert them first.

- There are 2 ways to convert
 - stringstream objects
 - string library functions

stringstream conversion

- ostreamstream
 - <<
 - to a string
- istreamstream
 - >>
 - from a string

Examples


```
1 //simpSStream.cpp
2 #include <iostream>
3 #include <sstream> // For std::stringstream
4 using namespace std;
5
6 int main() {
7     stringstream ss; // Create a stringstream object
8
9     // Add different types of data to the stringstream
10    ss << "The total price is: " << 29.99 << " dollars.";
11
12    // Extract the concatenated string from the stringstream
13    string result = ss.str();
14
15    // Output the result
16    cout << result << endl;
17
18    return 0;
19 }
```



```
1 //simpStream.cpp
2 #include <iostream>
3 #include <sstream> // For std::stringstream
4 using namespace std;
5
6 int main() {
7     stringstream ss; // Create a stringstream object
8
9     // Add different types of data to the stringstream
10    ss << "The total price is: " << 29.99 << " dollars.";
11
12    // Extract the concatenated string from the stringstream
13    string result = ss.str();
14
15    // Output the result
16    cout << result << endl;
17
18    return 0;
19 }
```

Output:

The total price is: 29.99 dollars.

```

1 // program12-8.cpp -- illustrate the use of sstream objects.
2 #include <sstream>
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main() {
8     string str = "John  20 50";
9     const char *cstr = "Amy 30 42";
10    istringstream istr1(str);
11    istringstream istr2;
12    ostringstream ostr;
13
14    string name;
15    int score1, score2, average_score;
16
17    // Read name and scores and computer average then write ostr
18    istr1 >> name >> score1 >> score2;
19    average_score = (score1 + score2)/2;
20    ostr << name << " has average score " << average_score << "\n";
21
22    // set istr2 to read from the C string and repeat the above
23    istr2.str(cstr);
24    istr2 >> name >> score1 >> score2;
25    average_score = (score1 + score2)/2;
26    ostr << name << " has average score " << average_score << "\n";
27
28    // switch to hex output
29    ostr << hex;
30    // Write Amy's scores in hex
31    ostr << name << "'s scores in hex are: " << score1
32        << " and " << score2 << "\n";
33
34    // extract the str from ostr and print
35    cout << ostr.str();
36
37    return 0;
38 }

```

```

1 // program12-8.cpp -- illustrate the use of sstream objects.
2 #include <sstream>
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main() {
8     string str = "John  20 50";
9     const char *cstr = "Amy 30 42";
10    istringstream istr1(str);
11    istringstream istr2;
12    ostringstream ostr;
13
14    string name;
15    int score1, score2, average_score;
16
17    // Read name and scores and computer average then write ostr
18    istr1 >> name >> score1 >> score2;
19    average_score = (score1 + score2)/2;
20    ostr << name << " has average score " << average_score << "\n";
21
22    // set istr2 to read from the C string and repeat the above
23    istr2.str(cstr);
24    istr2 >> name >> score1 >> score2;
25    average_score = (score1 + score2)/2;
26    ostr << name << " has average score " << average_score << "\n";
27
28    // switch to hex output
29    ostr << hex;
30    // Write Amy's scores in hex
31    ostr << name << "'s scores in hex are: " << score1
32        << " and " << score2 << "\n";
33
34    // extract the str from ostr and print
35    cout << ostr.str();
36
37    return 0;
38 }

```

Output:

```

John has average score 35
Amy has average score 36
Amy's scores in hex are: 1e and 2a

```

- But can't we perform concatenation on a string by using the plus operator?
- Ex “hi” + “3.2” + world

- But can't we perform concatenation on a string by using the plus operator?
- Ex “hi” + “3.2” + world

Yes, but everything would need to be converted to a string first, then we could concatenate.

```
1 //simpConcat.cpp
2 #include <iostream>
3 #include <string> // For std::string and std::to_string
4 using namespace std;
5
6 int main() {
7     string result = "The total price is: " + to_string(29.99) + " dollars.";
8
9     // Output the result
10    cout << result << endl;
11
12    return 0;
13 }
```

```
1 //simpConcat.cpp
2 #include <iostream>
3 #include <string> // For std::string and std::to_string
4 using namespace std;
5
6 int main() {
7     string result = "The total price is: " + to_string(29.99) + " dollars.";
8
9     // Output the result
10    cout << result << endl;
11
12    return 0;
13 }
```

Output:

The total price is: 29.990000 dollars.

Another way to convert (C11 conversion functions)

to_string() functions

to_string(int)

to_string(long)

to_string(double)

stoX() functions

int stoi(string)

long stol(string)

float stof(string)

double stod(string)


```

1 // 12-9.cpp stoX() demo
2 #include <string>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     string str;
8     size_t pos;    // holds position of stop character
9
10    str = "-342.57is a number";
11    cout << "The string is " << str << endl;
12    double d = stod(str, &pos);
13
14    // convert string to double
15    cout << "The converted double is " << d << endl;
16    cout << "The stopping character is " << str[pos]
17         << " at position " << pos << endl;
18
19    // convert string to int
20    str = "-342.57is a number";
21    cout << "\nThe string is " << str << endl;
22    int i = stoi(str, &pos);
23    cout << "The converted integer is " << i << endl;
24    cout << "The stopping character is " << str[pos]
25         << " at position " << pos << endl;
26
27    // Convert string to int (binary)
28    str = "01110binary number";
29    cout << "\nThe string is " << str << endl;
30    i = stoi(str, &pos, 2);
31    cout << "The converted binary integer is " << i << endl;
32    cout << "The stopping character is " << str[pos]
33         << " at position " << pos << endl;
34
35    return 0;
36 }

```

```

1 // 12-9.cpp stoX() demo
2 #include <string>
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     string str;
8     size_t pos;    // holds position of stop character
9
10    str = "-342.57is a number";
11    cout << "The string is " << str << endl;
12    double d = stod(str, &pos);
13
14    // convert string to double
15    cout << "The converted double is " << d << endl;
16    cout << "The stopping character is " << str[pos]
17         << " at position " << pos << endl;
18
19    // convert string to int
20    str = "-342.57is a number";
21    cout << "\nThe string is " << str << endl;
22    int i = stoi(str, &pos);
23    cout << "The converted integer is " << i << endl;
24    cout << "The stopping character is " << str[pos]
25         << " at position " << pos << endl;
26
27    // Convert string to int (binary)
28    str = "01110binary number";
29    cout << "\nThe string is " << str << endl;
30    i = stoi(str, &pos, 2);
31    cout << "The converted binary integer is " << i << endl;
32    cout << "The stopping character is " << str[pos]
33         << " at position " << pos << endl;
34
35    return 0;
36 }

```

Output:

```

The string is -342.57is a number
The converted double is -342.57
The stopping character is i at position 7

The string is -342.57is a number
The converted integer is -342
The stopping character is . at position 4

The string is 01110binary number
The converted binary integer is 14
The stopping character is b at position 5

```

Where are these string
functions found?

Where are these string functions found?

Reference : <string> : string

VS.

Reference : <string>

Which one to use?

Use the stream operators when you need heavy duty conversion. These can handle converting into other bases. You need to create a stream object however.

Use the C11 conversion functions when you need some quick and dirty converting.

12.4 Writing Your Own C-String Handling Functions

- Not uncommon given all the problems with the ASCII table.


```
1 // 12-10.cpp -- C-String copy to array
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 void stringCopy(char [], const char []);
7
8 int main() {
9     const int S_LENGTH = 30;
10    char dest[S_LENGTH], source[S_LENGTH];
11
12    cout << "Enter a string up to " << S_LENGTH - 1
13         << " characters: ";
14    cin.getline(source, S_LENGTH);
15
16    stringCopy(dest, source);
17    cout << "The string you entered is:\n" << dest << endl;
18
19    return 0;
20 }
21
22 void stringCopy(char destStr[], const char sourceStr[]) {
23     int index = 0;
24     while (sourceStr[index] != '\0') {
25         destStr[index] = sourceStr[index];
26         index++;
27     }
28     destStr[index] = '\0';
29 }
```



```

1 // 12-10.cpp -- C-String copy to array
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 void stringCopy(char [], const char []);
7
8 int main() {
9     const int S_LENGTH = 30;
10    char dest[S_LENGTH], source[S_LENGTH];
11
12    cout << "Enter a string up to " << S_LENGTH - 1
13         << " characters: ";
14    cin.getline(source, S_LENGTH);
15
16    stringCopy(dest, source);
17    cout << "The string you entered is:\n" << dest << endl;
18
19    return 0;
20 }
21
22 void stringCopy(char destStr[], const char sourceStr[]) {
23     int index = 0;
24     while (sourceStr[index] != '\0') {
25         destStr[index] = sourceStr[index];
26         index++;
27     }
28     destStr[index] = '\0';
29 }

```

```
➤ ./main
```

```

Enter a string up to 29 characters: Now is the time for all good
The string you entered is:
Now is the time for all good

```

```

1 // 12-11.cpp -- C-String slice off first word
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 void nameSlice(char []);
7
8 int main() {
9     const int S_LENGTH = 41;
10    char name[S_LENGTH];
11
12    cout << "Enter your first and last name, "
13         << "separated with a space.\n";
14    cin.getline(name, S_LENGTH);
15
16    nameSlice(name);
17    cout << "Your first name is:\n" << name << endl;
18
19    return 0;
20 }
21
22 void nameSlice(char userName[])
23 { // Set first space to null terminator.
24     int index = 0;
25     while (userName[index] != ' ' &&
26           userName[index] != '\0')
27         index++;
28
29     if (userName[index] == ' ')
30         userName[index] = '\0';
31 }

```

```

1 // 12-11.cpp -- C-String slice off first word
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 void nameSlice(char []);
7
8 int main() {
9     const int S_LENGTH = 41;
10    char name[S_LENGTH];
11
12    cout << "Enter your first and last name, "
13         << "separated with a space.\n";
14    cin.getline(name, S_LENGTH);
15
16    nameSlice(name);
17    cout << "Your first name is:\n" << name << endl;
18
19    return 0;
20 }
21
22 void nameSlice(char userName[])
23 { // Set first space to null terminator.
24     int index = 0;
25     while (userName[index] != ' ' &&
26           userName[index] != '\0')
27         index++;
28
29     if (userName[index] == ' ')
30         userName[index] = '\0';
31 }

```

```

Enter your first and last name, separated with a space.
Professor Fowler
Your first name is:
Professor

```

```

1 // 12-12.cpp -- C-String w/o length parameter
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 int countChars(const char*, char);
7
8 int main() {
9     const int S_LENGTH = 51;
10    char userString[S_LENGTH];
11    char letter;
12
13    cout << "Enter a string up to " << S_LENGTH - 1
14         << " characters: ";
15    cin.getline(userString, S_LENGTH);
16    cout << "Enter a char and I'll tell you how many\n"
17         << " times it appears in the string: ";
18    cin >> letter;
19
20    cout << letter << " appears "
21         << countChars(userString, letter) << " times.\n";
22
23    return 0;
24 }
25
26 int countChars(const char* strPtr, char ch) {
27     int count = 0;
28     while (*strPtr != '\0') {
29         if (*strPtr == ch)
30             count++;
31         strPtr++;
32     }
33     return count;
34 }

```



```

1 // 12-12.cpp -- C-String w/o length parameter
2 #include <iostream>
3 using namespace std;
4
5 // Function Prototype
6 int countChars(const char*, char);
7
8 int main() {
9     const int S_LENGTH = 51;
10    char userString[S_LENGTH];
11    char letter;
12
13    cout << "Enter a string up to " << S_LENGTH - 1
14          << " characters: ";
15    cin.getline(userString, S_LENGTH);
16    cout << "Enter a char and I'll tell you how many\n"
17          << " times it appears in the string: ";
18    cin >> letter;
19
20    cout << letter << " appears "
21          << countChars(userString, letter) << " times.\n";
22
23    return 0;
24 }
25
26 int countChars(const char* strPtr, char ch) {
27     int count = 0;
28     while (*strPtr != '\0') {
29         if (*strPtr == ch)
30             count++;
31         strPtr++;
32     }
33     return count;
34 }

```

Enter a string up to 50 characters: Starting out with c++
Enter a char and I'll tell you how many
times it appears in the string: t
t appears 4 times.

Peer Review

Programming Exercise

Exercise

Exercise Goals

- Increase Topic Mastery
- Construct Knowledge
- Give you data to evaluate how well you are learning.

Programming Exercise

- Create a program with 3 functions: foo, bar, and bah. Each module has a constant C-string indicating what module is called (“I am foo”, “I am bar”, etc). There are no cout statements in these modules.
- Main() will display this string in sOut (which can only appear in 1 line).



What's this section
about?

Agenda

Ch 12 - Strings

12.1 C-Strings

12.2 Library Functions for Working with C- Strings

12.3 Conversions between Numbers and Strings

12.4 Writing Your Own C-String Handling Functions

12.5 More about the C++ String Class



NEXT TIME

Ch 11 - More OOP

11.1 The this pointer and Constant member Functions

11.2 Static Members

11.3 Friends of Classes

11.4 Memberwise Assignment

11.5 Copy Constructors

11.6 Operator Overloading

11.7 Rvalue References and Move Operations

11.8 Function Objects and Lambda Expressions

11.9 Type Conversion Operators

11.10 Convergent Constructors

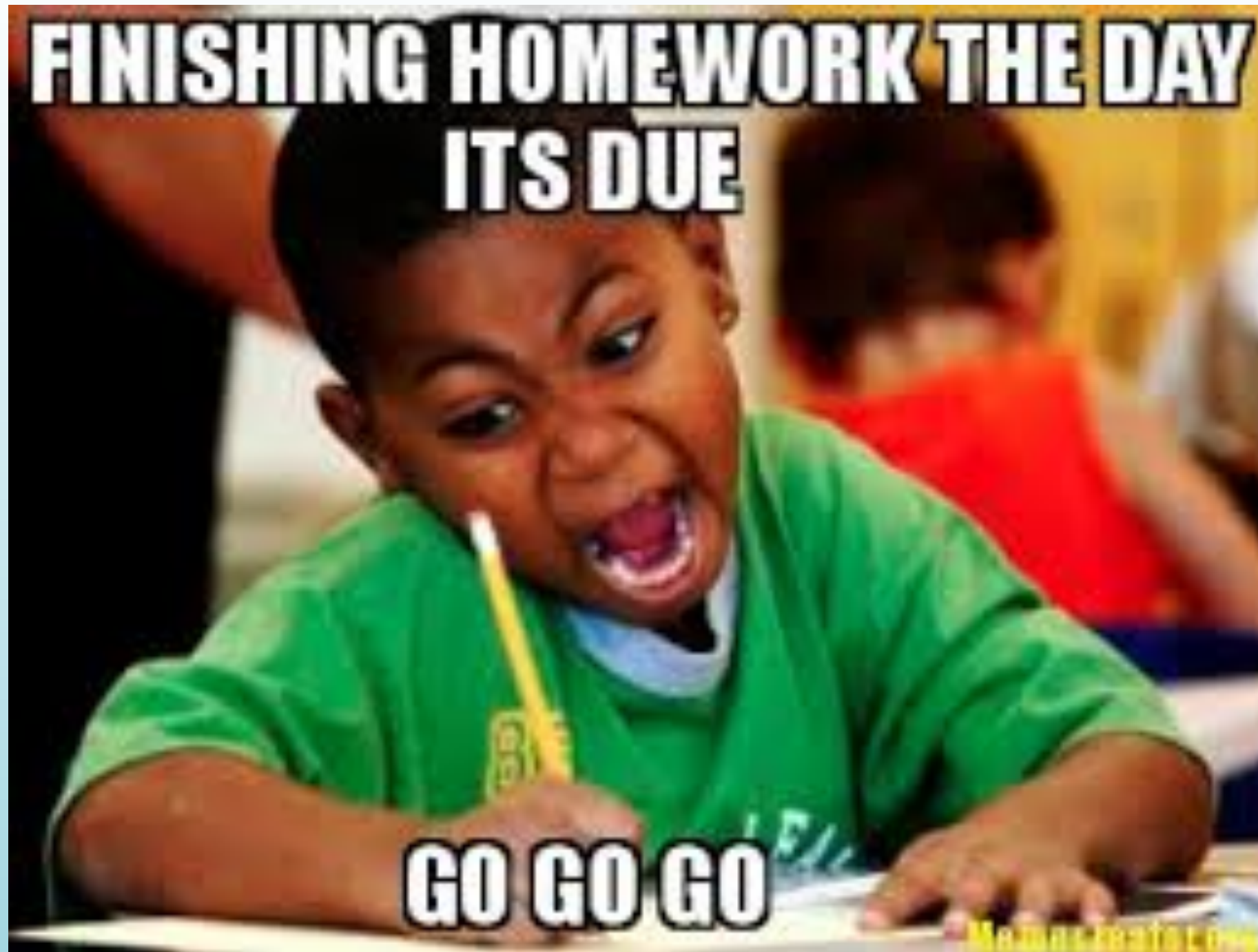
11.11 Aggregation and Composition

11.12 Inheritance

11.13 Protected Members and Class Access

11.14 Constructors, Destructors, and Inheritance

11.15 Overriding Base Class Functions



Homework 2 Peer Review due Sunday.

**HOW ARE WE
SUPPOSED TO SAMPLE THIS?**

**WHY IS SHE
WEARING GLOVES?**

Homework Preflight

Hw3 - InvInq

Exercise Goals

- Go over the homework and check understanding.
- Get questions answered.
- Discuss potential designs.

Task

- Read the next homework due in Canvas.
- Write down any questions you have about it.

Task - II

- Pair Up
- Go over each other's questions & see if you can answer them.

Task - III

- Ask me any unanswered questions.

A large, stylized yellow lightning bolt graphic is positioned diagonally across the center of the slide. It has a jagged, stepped appearance with multiple sharp points and turns, pointing from the upper right towards the lower left.

Lightning Review

```

f DispArr
1 // AODs.cpp -- Array of Dates
2 #include<iostream>
3 using namespace std;
4 class Date {
5     private:
6         int mo;
7         int day;
8         int year;
9     public:
10        Date(int = 1, int = 1, int = 1);
11        void PrintDate();
12 };
13 Date::Date(int m, int d, int y) {
14     mo = m;
15     day = d;
16     year = y;
17     if (mo > 12) mo = 99; // 99 is error SetDate
18     // other checking goes here
19 }
20 void Date::PrintDate() {
21     cout << mo << "/" << day << "/" << year << endl;
22 }
23
24 void DispArr(Date[], int);
25
26 // Basic array example
27 int main() {
28     Date dArr[2]{};
29
30     DispArr(dArr, 2);
31
32     // no cleanup - array on stack
33
34     return 0;
35 }
36
37 void DispArr(Date arr[], int s) {
38     for (int i = 0; i < s; i++)
39         arr[i].PrintDate();
40 }

```

```

f DispArr
1 // AODs.cpp -- Array of Dates
2 #include<iostream>
3 using namespace std;
4 class Date {
5     private:
6         int mo;
7         int day;
8         int year;
9     public:
10        Date(int = 1, int = 1, int = 1);
11        void PrintDate();
12 };
13 Date::Date(int m, int d, int y) {
14     mo = m;
15     day = d;
16     year = y;
17     if (mo > 12) mo = 99; // 99 is error SetDate
18     // other checking goes here
19 }
20 void Date::PrintDate() {
21     cout << mo << "/" << day << "/" << year << endl;
22 }
23
24 void DispArr(Date[], int);
25
26 // Basic array example
27 int main() {
28     Date dArr[2]{};
29
30     DispArr(dArr, 2);
31
32     // no cleanup - array on stack
33
34     return 0;
35 }
36
37 void DispArr(Date arr[], int s) {
38     for (int i = 0; i < s; i++)
39         arr[i].PrintDate();
40 }

```

```

./aod.ch
1/1/1
1/1/1

```



```

1  // AODs1.cpp -- Array of Dates
2  #include<iostream>
3  using namespace std;
4  ▶ class Date { ... };
13 ▶ Date::Date(int m, int d, int y) { ... }
20 ▼ void Date::PrintDate() {
21      cout << mo << "/" << day << "/" << year << endl;
22  }
23
24  void DispArr(Date*[], int);
25
26  // Array on stack, objects on heap
27 ▼ int main() {
28      Date* dArr[2]{};
29      for (int i = 0; i < 2; i++)
30          dArr[i] = new Date(i, i+2, 2000+1);
31
32      DispArr(dArr, 2);
33
34  // cleanup - Array on stack, objects on heap
35      delete dArr[0];          // each new sb balanced w/ a delete
36      delete dArr[1];
37      dArr[0] = nullptr;
38      dArr[1] = nullptr;
39
40      return 0;
41  }
42
43 ▼ void DispArr(Date* arr[], int s) {
44      for (int i = 0; i < s; i++)
45          arr[i]->PrintDate();
46  }

```



```

1  // A0Ds1.cpp -- Array of Dates
2  #include<iostream>
3  using namespace std;
4  ▶ class Date { ... };
13 ▶ Date::Date(int m, int d, int y) { ... }
20 ▼ void Date::PrintDate() {
21      cout << mo << "/" << day << "/" << year << endl;
22  }
23
24  void DispArr(Date*[], int);
25
26  // Array on stack, objects on heap
27 ▼ int main() {
28      Date* dArr[2]{};
29      for (int i = 0; i < 2; i++)
30          dArr[i] = new Date(i, i+2, 2000+1);
31
32      DispArr(dArr, 2);
33
34  // cleanup - Array on stack, objects on heap
35      delete dArr[0];          // each new sb balanced w/ a delete
36      delete dArr[1];
37      dArr[0] = nullptr;
38      dArr[1] = nullptr;
39
40      return 0;
41  }
42
43 ▼ void DispArr(Date* arr[], int s) {
44      for (int i = 0; i < s; i++)
45          arr[i]->PrintDate();
46  }

```

```

➤ make -s
➤ ./main
0/2/2001
1/3/2001
➤

```

```

1  // AODs2.cpp -- Array of Dates
2  #include<iostream>
3  using namespace std;
4  ▶ class Date { ... };
13 ▶ Date::Date(int m, int d, int y) { ... }
20 ▶ void Date::PrintDate() { ... }
23
24  void DispArr(Date*[], int);
25
26  // pointer on stack, array & objects on heap
27 ▼ int main() {
28      Date** dArr = new Date*[2];           // changed
29      for (int i = 0; i < 2; i++)
30          dArr[i] = new Date(i, i+2, 2000+1); // also works: dArr[i] = new Date;
31
32      DispArr(dArr, 2);
33
34      // cleanup - pointer on stack, array & objects on heap
35      // each new sb balanced w/ a delete
36      delete dArr[0];
37      delete dArr[1];
38      dArr[0] = nullptr;                     // we can discuss if this is really needed
39      dArr[1] = nullptr;                     // we can discuss if this is really needed
40      delete [] dArr;                        // new
41      dArr = nullptr;                       // new
42
43      return 0;
44  }
45
46 ▼ void DispArr(Date* arr[], int s) {
47     for (int i = 0; i < s; i++)
48         arr[i]->PrintDate();
49 }

```

```

1  // AODs2.cpp -- Array of Dates
2  #include<iostream>
3  using namespace std;
4  ▶ class Date { ... };
13 ▶ Date::Date(int m, int d, int y) { ... }
20 ▶ void Date::PrintDate() { ... }
23
24  void DispArr(Date*[], int);
25
26  // pointer on stack, array & objects on heap
27 ▼ int main() {
28      Date** dArr = new Date*[2];           // changed
29      for (int i = 0; i < 2; i++)
30          dArr[i] = new Date(i, i+2, 2000+1); // also works: dArr[i] = new Date;
31
32      DispArr(dArr, 2);
33
34      // cleanup - pointer on stack, array & objects on heap
35      // each new sb balanced w/ a delete
36      delete dArr[0];
37      delete dArr[1];
38      dArr[0] = nullptr;                     // we can discuss if this is really needed
39      dArr[1] = nullptr;                     // we can discuss if this is really needed
40      delete [] dArr;                        // new
41      dArr = nullptr;                        // new
42
43      return 0;
44  }
45
46 ▼ void DispArr(Date* arr[], int s) {
47     for (int i = 0; i < s; i++)
48         arr[i]->PrintDate();
49 }

```

```

➤ make -s
➤ ./main
1/1/1
1/1/1

```

Class Starters

Exercise

What's the difference ...

- `char* = "This is a test";`
`char a[] = "This is a test\0";`

Sentence Starters

- The question I'd like most to ask the author is...
- The idea I most take issue with from the text is ...
- The most critical point from last week's class discussion was ...
- The idea I found most interesting was ...
- The part of the class session I found most confusing was ...
- The part of the class session which surprised me the most was ...
- The idea that helped me relate the class session to what I already know about the topic was ...